IS5126 Assignment 1

DATA FOUNDATION AND EXPLORATORY ANALYTICS

Group 1

Chan Chi-Loong	A0318138A
Nadia Victoria Aritonang	A0314698N
Reiner Anggriawan Jasin	A0314502W
Tran Thien Nhan	A0200690R
Yeo Zhuan Yu	A0168723X

Repository: https://github.com/nadiavictoria/IS512-Hotel-Analytics

Report repository and team member contribution

Repository: https://github.com/nadiavictoria/IS512-Hotel-Analytics

Name	Contribution	Gen AI use
Chan Chi-Loong	• Competitive benchmarking and business questions • Notebook 3 and segment 5 in report	VSC copilot
Nadia Victoria Aritonang	• Data preprocessing and data foundations design • Notebook 1, data-preprocessing.py and segment 2 in report	ChatGPT Full prompt history can be accessed here
Reiner Anggriawan Jasin	• Exploratory Data Analysis and Feature Engineering • Notebook 2, and segment 3 in report	Gemini Link in Notebook
Tran Thien Nhan	• Non-technical Dashboard • Streamlit_app.py file and segment 6 in report	Gemini Full prompt history can be accessed here
Yeo Zhuan Yu	• Performance Profiling and Optimization • Notebook 4, and segment 4 in report	ChatGPT Full prompt history can be accessed here

All members contributed to the report and coding.

1. Executive Summary

This project develops an intelligent analytics platform to help hotel managers understand the current hospitality landscape and make appropriate data-drive decisions. The analytics platform is built based on a large-scale hotel review data, which is then preprocessed and stored into an SQL database. The reviews were then analyzed through EDA and competitive benchmarking. Following this, the database is profiled for enhanced storage and retrieval. Finally, important features derived from the analysis are displayed in an interactive dashboard.

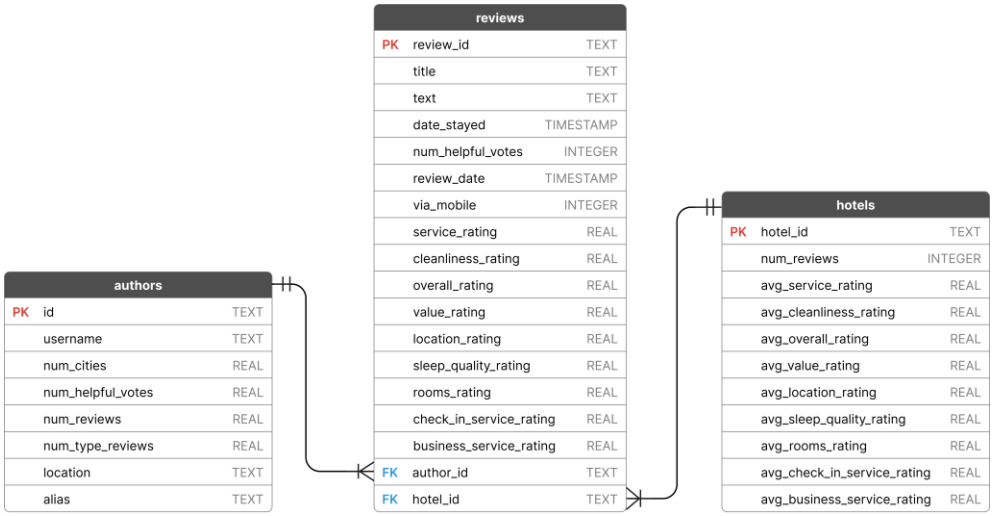
2. Data foundation

2.1. Raw Data

The original JSON file contained 878.561 records that each represented a singular hotel review. Each review provides numerical ratings, textual descriptions, author metadata, and temporal data. A review is written by one author for one hotel. On the other hand, a hotel can receive multiple reviews, and an author can write multiple reviews. Therefore, the relationship between hotels-to-reviews can be categorized as a one-to-many relationship. The same applies for the authors-to-reviews relationship.

2.2. Database Design

Based on the entity relationships, we decided to create 3 database entities: **review, author,** and **hotel**. An advantage of this design decision is that it reduces redundancy. For example, when an author writes multiple reviews, the author metadata will only be contained in a singular data entry while the unique reviews will be stored in their own individual records. The overall database schema is pictured below.



2.3. Data Preprocessing and Filtering

The raw data was split into the review, author, and hotel dataset according to the schema. Before inputting the data into the database, the data was preprocessed and filtered to ensure data consistency and quality. The following paragraphs will go into further detail on how each dataset was processed.

The reviews were first filtered to only include the most recent five years (2008-2012). Reviews that were written after more than 1 month were also discarded. Both of these measures were done to ensure relevancy towards the more current conditions of the hotels. Invalid reviews that had review dates before the date of stay were also removed. Next, missing values were removed for all columns except for sleep quality ratings, check-in service ratings, and business service ratings.

These ratings were excluded from removal because sleep quality ratings were newly added in 2010 while check-in and business service ratings were only filled in 2008. These ratings were not dropped in the initial phase to comprehensively evaluate the data and analyze any potential insights before completely excluding them. Finally, duplicates were handled by removing exact duplicates and selecting the earliest versions for reviews that had identical titles and texts.

The author dataset was then processed by removing authors with missing IDs. Authors with "CATID_", which is assumed to be an invalid ID, were also removed. Next, authors that had multiple usernames under the same user ID were consolidated into 1 record. In this case, the most recent entry was retained to reflect the author's most up-to-date data, while saving the older usernames in a new alias column.

Lastly, the hotel dataset was extracted from the hotel_id column (previously called listing_id, renamed for clarity) from the review dataset. Aside from hotel ID, we also derived the number of reviews received for each hotel and the average of each of its ratings.

2.4. Sampling and Indexing

After filtering and preprocessing, the dataset still had around 500.000 reviews. Therefore, the dataset was randomly sampled for 80,000 reviews for the entire analysis database, resulting in the *review_analysis.db* file. Furthermore, the analysis database was then randomly sampled for 6,000 reviews to create the sample database (*review_sample.db)*. The hotel and authors included in the database were also purged to remove instances that are not present in the reviews.

Lastly, indexes were placed in the database to make queries more efficient, noting that the project handles a huge amount of data. Each identifier (review ID, author ID, and hotel ID) was uniquely indexed for data integrity. Other indexes were added to anticipate and support frequent queries. This includes a hotel id index in the review dataset for selecting specific hotels, a review date index in the review dataset for analyzing trends over time, and a composite index for hotel id and review date that optimizes queries on both identifiers.

3. Exploratory Data Analytics

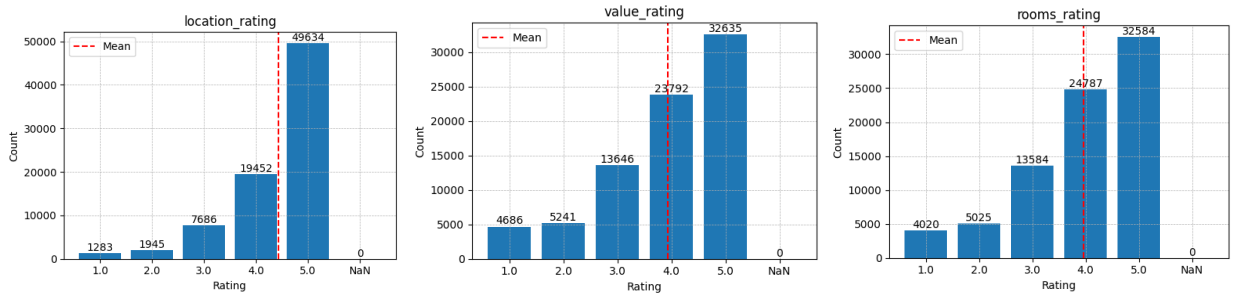
3.1. Review Table Data Exploration

Because our goal is to evaluate hotel performance to help the hotel build a better strategy for quality improvement, this report will focus on the review table, with important insights from other tables mentioned occasionally. We started off by checking the data type and ensuring that the datetime entries are parsed correctly. After checking for missing values and removing invalid entries, the dataset leaves the whole 80,000 data unchanged, showing that the data is already clean for further checking.

To check the review data validity and relevancy. A review is considered invalid if any required information (hotel_id, author_id, date_stayed, or review_date) is missing or if the review_date occurred before the date_stayed, For the data relevancy, we restricted the data to reviews posted within one month of their stay duration. During iterations, we found some authors who reviewed the hotel years after their stay date. So, to keep the data relevant and prevent recall bias, we decided to pick 1 month as the window filter which still gives us sufficient data to process.

The descriptive statistics analysis (Table 3.1 and Table 3.2) shows that most rating categories contain 80,000 entries. However, three columns have recently been added by TripAdvisor, and they contain a significantly fewer entry. The mean of our column is higher than 3 stars, indicating positivity bias which is common in reviews dataset. The mean ranges from 3.67 stars for business service rating and 4.43 stars for location rating. Because hotels receive multiple reviews, these means are actually weighted toward hotels with a higher amount of customers. When using the equal-weighted rating across hotels (hotels_df), the average slightly drops and shows lower variance, which confirms that hotels with many reviews can inflate the overall mean.

3.2. Review Rating Distribution

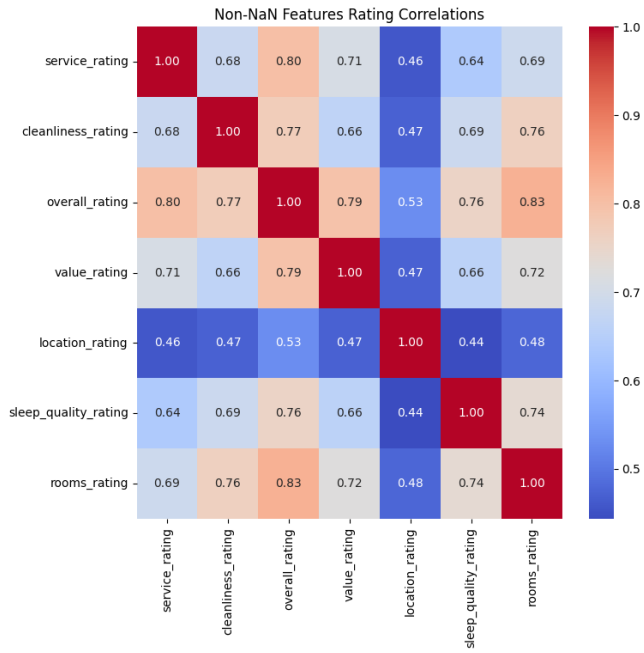


Ratings grouped by stars plot (Figure 3.1), show a skew towards higher ratings which re-confirm the positivity bias we mentioned above. The location rating receives the highest amount of 5-star, with value and room ratings showing relatively lower ratings as seen in (Figure 3.2). From a hotel owner perspective, this observation can help them in making decisions about the hotel improvement plan, as improvements in price value and room quality can produce greater gain and increasing the competitive level of the hotel better than promoting location related features. In an overview, this observation now changes our perspective when looking at a 4-star rated category. Accurate interpretation cannot be omitted unless we know how the other hotels are performing.

For the remaining non-rating features (Figure 3.3), the distribution of helpful cotes is extremely right-skewed, showing the minimal number of reviews receiving more than five upvotes. The via_mobile feature shows that only 6,7& of authors review via mobile. This is important because, based on our further exploration, review via mobile allows the hotel to receive reviews sooner. This can help the hotel spot a trend change in the dataset and act accordingly before further negative impact. So, encouraging reviews via mobile may help hotels obtain more relevant and reliable feedback.

3.3. Feature Correlation

The correlation analysis (Figure 3.4) shows a strong correlation between rooms, service, and value ratings to the overall ratings. The room condition and service quality show the highest correlations, which indicates that these are the key factors of customer satisfaction. Meanwhile, location has the weakest correlation to the overall rating. This can happen because customers typically select hotels based on their preferred location before ordering, reducing the chance of picking a hotel located far beyond the customer's expectations and ended up leaving bad reviews regarding the location. The high positive correlations on our features encourage us to implement the Principal Component Analysis (PCA) for dimensionality reduction.



3.4. Hotels Time-based Trend

To look into the hotel performance over time, we aggregate the ratings by each month for one specific hotel denoted by the hotel_id (in this case the id is 214197) during the latest 5-year period of customer review. Our chart (Figure 3.5) shows that the trend across different ratings moves closely together, which explains their high correlations. In almost all of the categories, we can also observe how the hotel attempts to improve its quality by receiving more customers and gathering more reviews over time. This time-based analysis can help hotels detect performance volatility and adjust strategies accordingly. The same graph will be modified to compare the hotel performance to market analysis on the later part.

3.5. Best Vs. Worst Performer Hotel

To help us analyze the market conditions, we also tried to identify the best and worst performance from each category (Figure 3.6). We ranked the hotels with a multi-level sorting approach that is based on the rating (descending), number of reviews (descending), and rating variance (ascending). Additionally, we implement the minimum reviews threshold to filter out hotels with insufficient observations. Our approach is expected to show the top and bottom performing hotel in the market in a more reliable manner.

Our EDA comprises of many steps that cannot be compiled in such short report, additional analysis such as TF-IDF keyword extraction are excluded from this report. For this reason, we encourage the reader to go through our notebook for even more interesting findings.

4. Performance profiling and optimization

Performance profiling was conducted across two dimensions, query profiling at the database layer and code profiling at the Python application layer. This is to identify bottlenecks, validate the indexing strategy and measure the impact of optimizations on the 80,000 review analytical pipeline. Refer to Table 4.1 for the tools used.

4.1. Query Profiling

The profiling baseline corresponds to the analysis database after filtering and preprocessing. To isolate index effects from disk I/O variability, all 80,000 rows were loaded into two in-memory SQLite databases (Table 4.2 and Table 4.3).

- db_no: Tables only, no indexes (naive baseline)
- db_yes: Identical data plus the full index set

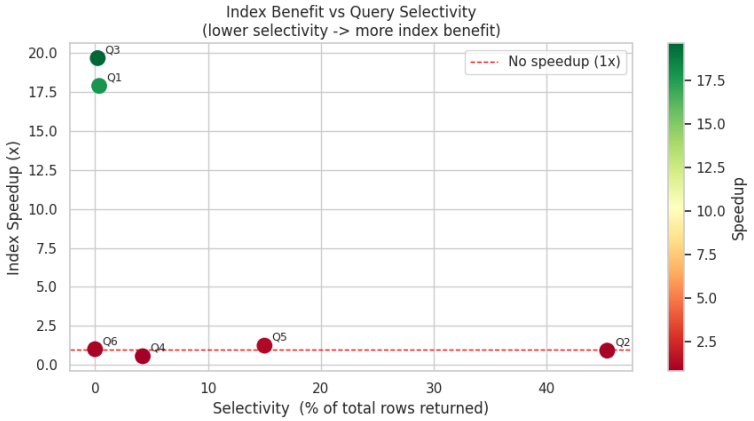
Each query was executed 12 times per configuration and median latency is reported to suppress JIT warm-up noise. The most reviewed heavy hotel (hotel_id = 2114197 with 295 reviews) was used as the point lookup test case.

EXPLAIN QUERY PLAN was run before benchmarking to confirm which access path the query planner would choose for each configuration (Table 4.4). The plan confirms that Q1 and Q3 benefit from direct B-tree seeks, Q5 is a nested-loop join accelerated on the probe side and Q6 cannot use any index because the % wildcard prefix prevents prefix matching.

4.1.1 Benchmark Results

The results divide clearly into two groups, queries that benefitted from indexes (Q1, Q3) and those that do not (Q2, Q4, Q6). This divergence is explained by index selectivity, which is the fraction of rows a query returns.

Selectivity is the single strongest predictor of index benefit. Queries that retrieve a small fraction of the table (Q1 at 0.37%, Q3 at 0.24%) benefit dramatically from B-tree index seeks. Queries that must touch a large fraction of rows (Q2 at 45.4%, Q4 at 4.2% but covering every hotel) incur overhead from pointer-chasing with no corresponding reduction in row reads.



Looking at the EXPLAIN QUERY PLAN for Q5, it shows that the index did not help the reviews side, since it must scan all rows to apply the filter on the hotels table. However, the unique index on hotels.hotel_id accelerates the join probe, for each of the 80,000 review rows, SQLite looks up the matching hotel via the B-tree rather than scanning all 3,380 hotel rows. This is where the small gain in speedup comes from.

Q2 did not benefit from indexes due to the selectivity threshold issue. The sweep over seven date cutoffs identifies the exact point at which the date index transitions from beneficial to harmful. The break-even point lies between approximately 25% and 45% selectivity. The actual Q2 query (45.4%) falls above this threshold, meaning that the index costs more than it save at this data volume. Narrowing the data window to a single month (1.9% selectivity) yields a 3.94% speedup from the same index.

Q4 did not benefit from indexes due to the full-coverage aggregation problem. The GROUP BY hotel_id COUNT + AVG(overall_rating) aggregation must read overall_rating from every row. The idx_reviews_hotel index stores only hotel_id, so a row heap-fetch is still required for each of the 80,000 rows. The index adds pointer-chasing overhead without reducing row reads, making it 0.56× as fast as a sequential scan. This is the worse index-overhead case observed. To test the remedy, a covering index on (hotel_id, overall_rating) was created.

The covering index answers the entire aggregation from the index structure alone, never touching the heap, resulting in a 9.49× improvement over the hotel_id-only index. However, this index is not added to the actual schema because the hotels table already stores pre-computed aggregates (avg_overall_rating, etc.), serving the same function at zero query-time cost.

For query anti-patterns, three common coding mistakes were benchmarked against their corrected forms. The correlated subquery case is the most impactful finding. The slow version executes a full sub-scan of the reviews table once per outer row, taking 3.47 seconds for 80,000 rows. Replacing it with a pre-aggregated derived table reduces this to 101 ms, a 34.5× improvement. Applying the function wrap fix alone restores the full 20.5× B-tree speedup for hotel lookup queries.

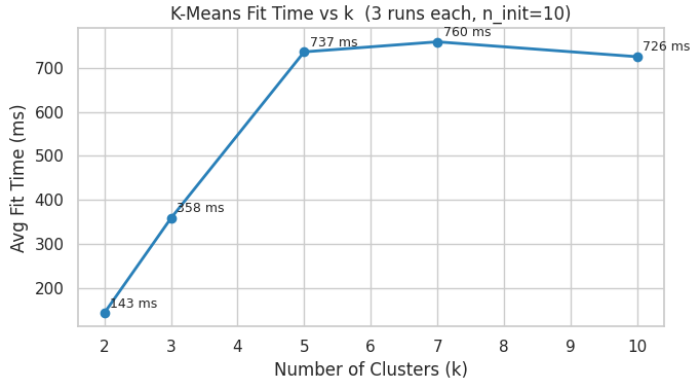
4.2. Code Profiling

Three pd.read_sql strategies were benchmarked across 3 runs each on the 80,000-row reviews table. Column projection delivers the most meaningful improvement, as fetching only 6 of 18 columns reduces load time from 367 ms to 157 ms (2.34× faster). Additionally, the memory profiling results show 7.1× reduction in heap allocation from 112.45 MB to 15.77 MB. Chunked loading is slower than a full load at this dataset size due to the pd.concat overhead per batch. It provides benefit only when the full dataset does not fit in RAM.

cProfile on pd.read_sql('SELECT * FROM reviews') reveals a 3,595-call trace totalling 0.437 seconds. The dominant cost (44%) is I/O polling, confirming that at this data volume the bottleneck is I/O scheduling rather than CPU computation. The 29% spent in _convert_arrays_to_dataframe is proportional to column count, meaning that column projection directly reduces this overhead by loading fewer columns.

For vectorisation, the comparative performance of the three methods used to compute the per-hotel mean overall rating was evaluated. Pandas groupby is 138× faster than iterrows on 10,000 rows and scales efficiently to 80,000 rows (4.96 ms). NumPy bincount is slower than Pandas at 54.70 ms on this dataset because np.unique applied to string-typed hotel_id values incurs substantial lexicographic sorting overhead. NumPy bincount would outperform Pandas only if hotel IDs were pre-encoded as contiguous integers.

The competitive benchmarking pipeline runs StandardScaler + KMeans over 3,380 hotels on 3 rating features. cProfile with n_init=10 and k=5 yields.
- Total: 0.758 seconds, 20,560 function calls
- Lloyd's algorithm: 0.668 s (88%) across 10 independent initialisations
- epoll async I/O: 0.043 s (6%), event loop overhead during parallel thread management



For memory profiling, the full SELECT * load allocates 112.45 MB, which is the dominant memory consumer in the pipeline, approximately 7.1× greater than the projected load at 15.77 MB. All post-load operations (aggregation, clustering) are negligible by comparison, confirming that data ingestion is the primary memory bottleneck, not the analytics computation.

5. Competitive benchmarking strategy

For this segment, we try to group hotels that are similar for benchmarking. From Siteminder hotel classifications, the main differences between the different hotels in terms of star rating (from 1 to 5 star) are services, room quality, location and cost. Hence, we narrowed down to this small list of features for clustering as they represent what we need to try to distinguish between the hotels: service rating, location rating, and room rating.

We considered and tested other features like **overall rating**, **value rating** and **cleanliness rating** (and even all the ratings) but ultimately settled on these 3 because the other ratings were either composites of others (e.g. **overall rating**) or highly correlated. This can be seen from the earlier exploratory analysis. Moreover, these 3 ratings had the most explanatory power for business analysis

We did some exploratory data analysis on the three features chosen to get an idea of the skewness (seemed okay), then did a K-means analysis to discover clusters. K-means, as compared to other methods like HDBSCAN, seemed relevant for this dataset because the dataset features were relatively clean and well defined. By testing k-means values between 2 and 10 and doing an elbow and silhouette analysis, the number of clusters chosen was **5,** which is a good number chosen for analyzing. Not too many clusters that would be difficult for business to look at.

Finally, we reduced the feature dimensions via **Principal Component Analysis (PCA)** to just 2D so it can be easily visualized, and we also did some analysis to see how our features contribute to PCA 1 (**location rating**) and PCA 2 (**rooms + service rating**).

Lastly, we did some cluster profiling so that we know what all these 5 clusters are. Here's the normalized compute indexes relative to each other:

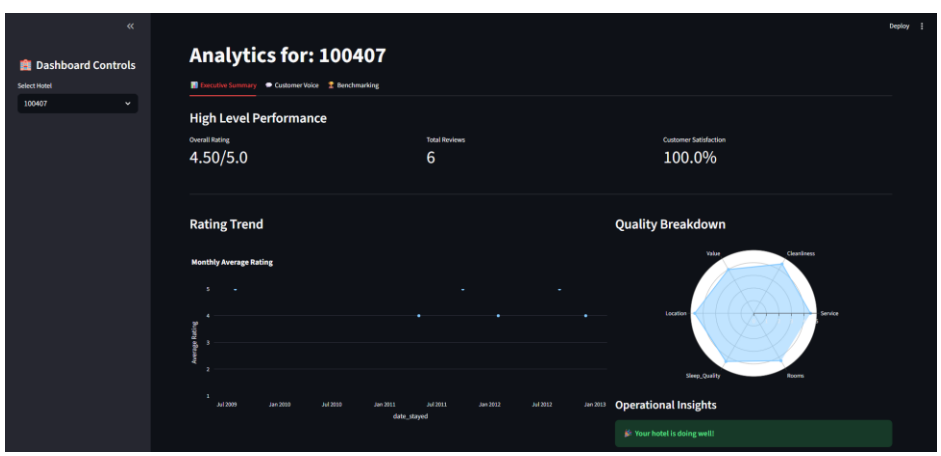
Cluster	Service rating	Location rating	Room rating	Percentage
0	115	111	118	36.7%
1	99	109	98	19.5%
2	80	86	76	15.1%
3	50	61	47	6.3%
4	103	94	103	22.4%

Here's the cluster analysis:

Cluster name	Description	Steps to take
Top of the line	These are the best hotels, with above average scores in every category. Likely at least a 3-star hotel, if not 4 or 5 star.	Continue to do what they are doing, and marketing should emphasize their excellent review scores.
Convenient	Below average, no-frills hotels that stand out because of their good location ratings.	Location is key for these hotels, and should be the most important feature in their marketing.
Below average	Hotels that are below average in every rating, but not as bad as the terrible hotels.	Improve service and room quality first before tackling convenience factor.
Terrible	These are the worst hotels, with especially low room and service ratings.	Reorganize the management of the hotel, then start improving room quality (with a score of 47, this is less than half the average of all hotels)
Good, but inconvenient	Above average hotels that are situated in more out of the way areas.	These hotels are harder to get to (e.g. beach resort), but are very good otherwise. Offering free transport options (hotel pick-up) can be a good way of enticing travelers to come here.

6. System architecture and dashboard

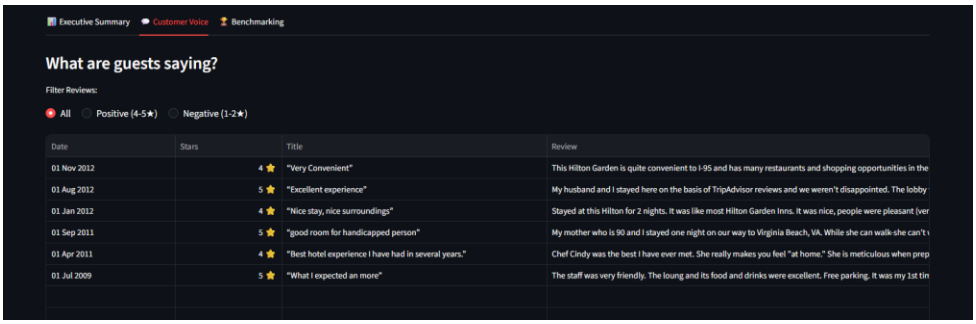
After conducting various data analyses, we created a streamlit dashboard to display key actionable insights. This dashboard aims to help hotel managers understand the performance of their hotels and make decisions to increase their competitiveness in the market. Since the target audience is non-technical, we try to make this dashboard as user-friendly as possible while displaying sufficient information. The goal is to display "insight" and not "data".



To use the dashboard, the hotel manager must first select their hotel id in the “Dashboard Controls” section. The dashboard will display all information that belongs to the selected hotel. The dashboard contains 3 pages: “Executive Summary”, “Custom Voice”, and “Benchmarking”. The Executive Summary displays the hotel’s high-level performance, including their rating, number of reviews, and percentage of satisfied customers (based on the number of positive reviews). The Rating Trend shows the monthly average rating over time to help managers identify performance gaps at any particular period.

The Quality Breakdown shows the performance of the hotel in 6 rating categories. If the average rating of any category falls below 2.5, the streamlit application will highlight it as a critical focus area. Any average rating from 2.5 to 4 will be highlighted as an area of improvement. If the average ratings of all 6 categories are above 4, the streamlit application will highlight that the hotel is doing well. The Executive Summary gives hotel managers a quick oversight of their hotel’s overall performance to make necessary high-level decisions.

The Customer Voice page shows all the reviews that the hotel received. Hotel managers can sort out the reviews by date, number of stars, or display positive versus negative reviews. The Customer Voice page allows hotel managers to really understand how their customers felt about the hotel and take specific actions to resolve any issue.



The Benchmarking page shows hotel managers how their hotels are performing in comparison to the market. Based on our benchmarking analysis results (section 5), the streamlit application displays the Cluster Map and where the selected hotel is located. By understanding which cluster group the hotel belongs to, hotel managers can adjust their marketing strategy that aligns with their competitive strengths and addresses their weaknesses.

The Market Comparison shows the selected hotel's ratings in the 6 categories compared to the market average ratings in those categories. The application will highlight the hotel's selling point and weakness in comparison to the market average.

7. Conclusion

We created a dashboard that allows end users to access the insights from the exploratory analysis without having to check the database directly. Through the development process, we identified that room quality and service ratings had an important influence on overall ratings. We also saw that clustering hotels segmented the hotels into identifiable profiles, allowing hotel managers to strategize based on their segments. Performance profiling also validated the database design decisions, showing that the indexing was effective.

There are some limitations and improvements that can be implemented in future works. Although the data was filtered to include the most recent years, the range of dates was only from 2008 until 2012. We also found that the database lacked features that could have been beneficial for our analysis, such as hotel location. The current iteration focuses on structured data types; however, we plan to integrate more advanced methods such as sentiment analysis and text analytics.

Appendix

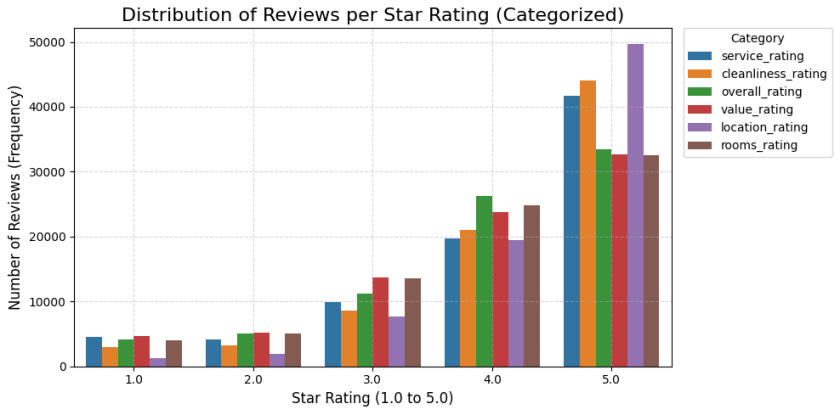


Figure 3.1. Number of Review per Star Rating Group

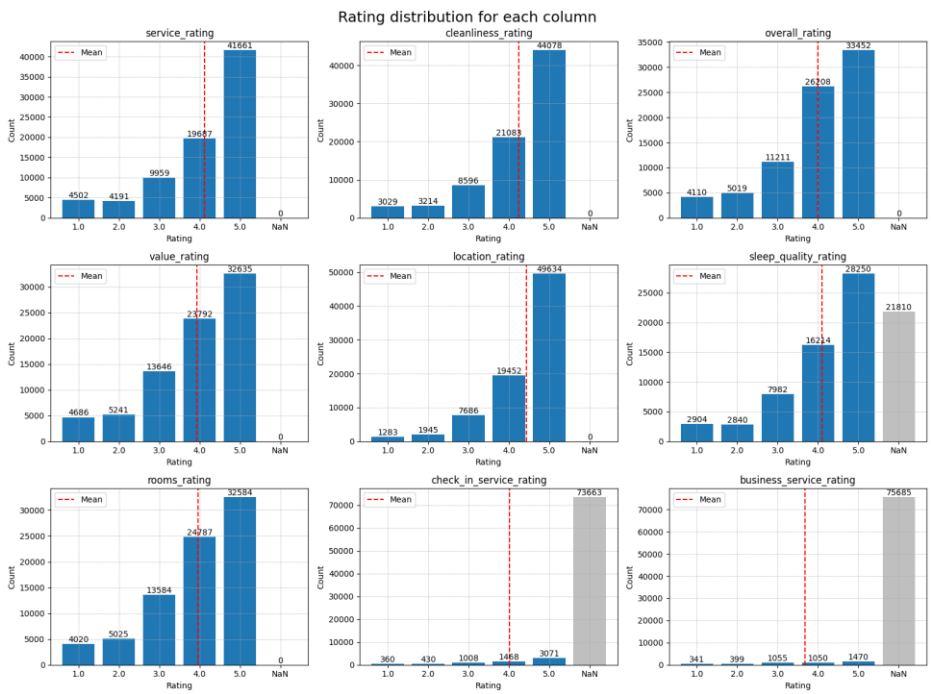


Figure 3.2. Star Rating Distribution per Category

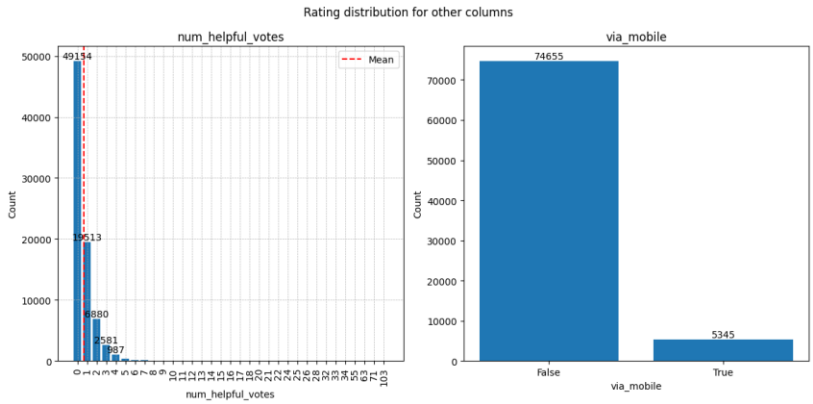


Figure 3.3. Non-rating Column Distribution

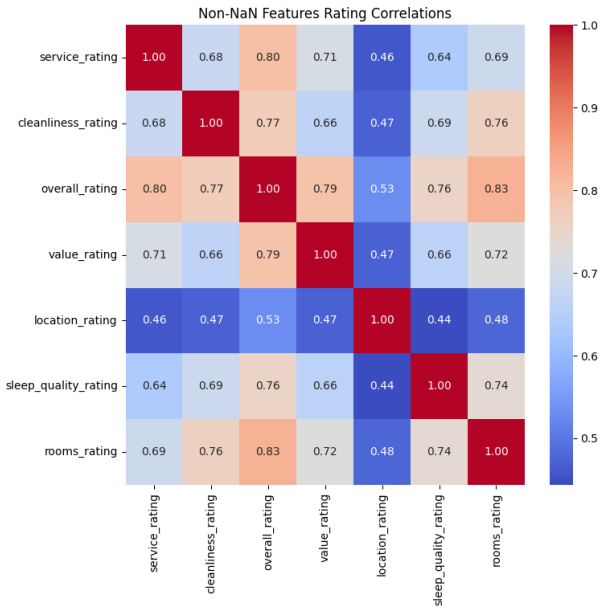


Figure 3.4. Non-NaN Feature Rating Correlation

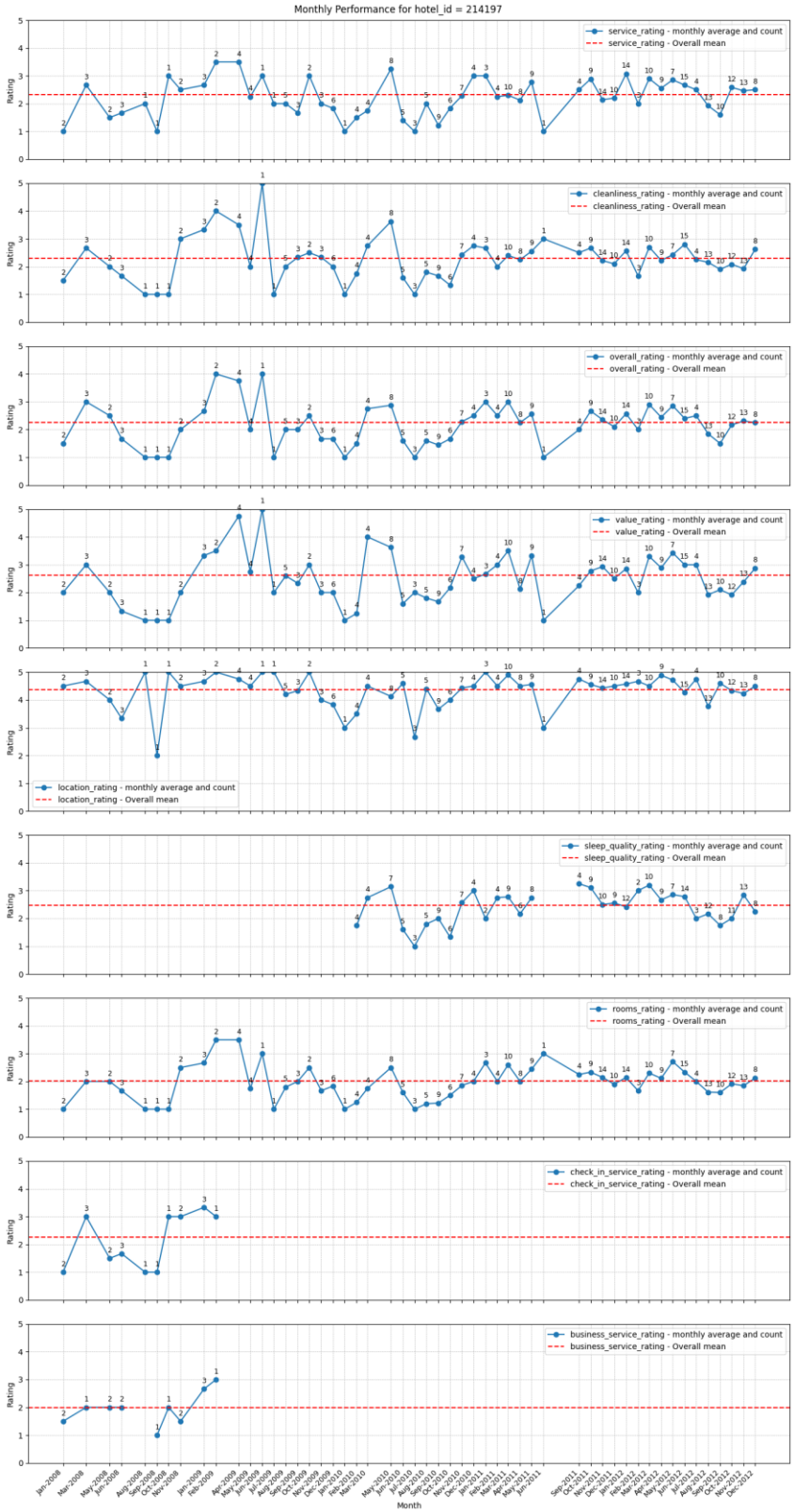


Figure 3.5. Time-based Hotel Monthly Performance

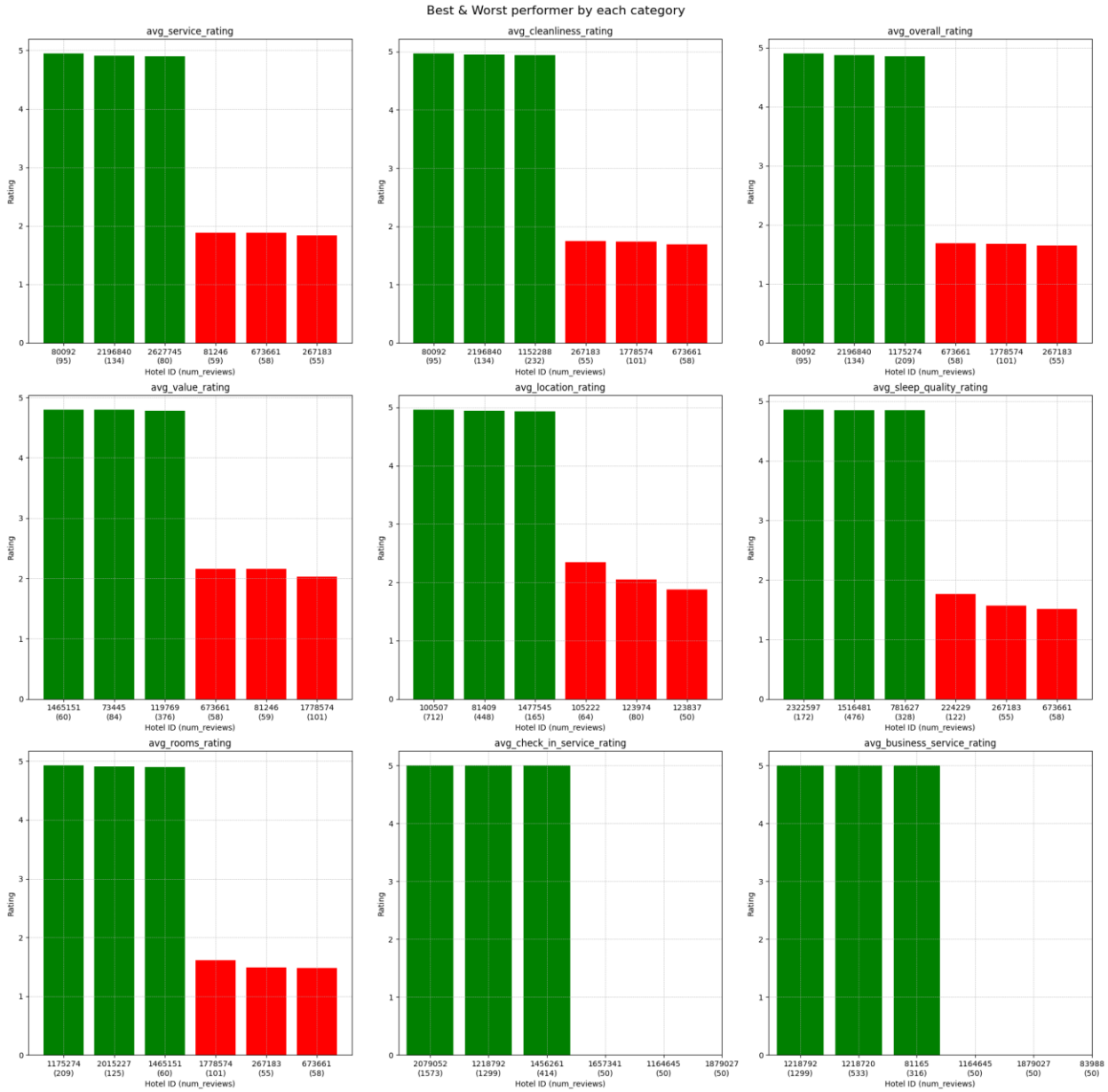


Figure 3.6. Best & Worst Hotel Performance

Table Name	Valid entry	Missing data	Date range	
			Date_stayed	Review_date
Reviews_df	80,000	0	Dec 2007 – Dec 2012	1 Jan 2008 – 20 Dec 2012
Authors_df	75,349	0	-	-
Hotels_df	3,380	0	-	-

Table 3.1. Database Analysis

Column name	Count	Mean	Std	min	Q1	Q2	Q3	max
Via_mobile	80,000	0.067	0.250	0	0	0	0	1
Service_rating	80,000	4.123	1.160	1	4	5	5	5
Cleanliness_rating	80,000	4.250	1.048	1	4	5	5	5
Overall_rating	80,000	3.998	1.128	1	3	4	5	5
Value_rating	80,000	3.931	1.167	1	3	4	5	5
Location_rating	80,000	4.428	0.880	1	4	5	5	5
Sleep_quality_rating	58,190	4.101	1.121	1	4	4	5	5
Rooms_rating	80,000	3.961	1.131	1	3	4	5	5
Check_in_service_rating	6,337	4.019	1.194	1	3	4	5	5
Business_service_rating	4,315	3.674	1.249	1	3	4	5	5

Table 3.2. Reviews_df Descriptive Statistics

Column name	Count	Mean	Std	min	Q1	Q2	Q3	max
Num_reviews	3,380	148.26	218.97	1	23	60.5	184	1825
Avg_service_rating	3,380	3.85	0.73	1	3.57	4.03	4.34	5
Avg_cleanliness_rating	3,380	3.90	0.81	1	3.60	4.14	4.47	5
Avg_overall_rating	3,380	3.65	0.79	1	3.33	3.83	4.20	5
Avg_value_rating	3,380	3.75	0.66	1	3.50	3.88	4.19	5
Avg_location_rating	3,380	4.06	0.65	1	3.75	4.19	4.53	5
Avg_sleep_quality_rating	3,347	3.79	0.74	1	3.50	3.99	4.29	5
Avg_rooms_rating	3,380	3.67	0.79	1	3.30	3.84	4.23	5
Avg_check_in_service_rating	2,503	3.83	0.87	1	3.45	4.00	4.44	5
Avg_business_service_rating	2,384	3.56	0.94	1	3.00	3.69	4.20	5

Table 3.3. Hotels_df Descriptive Statistics

Tool	Purpose
SQLite EXPLAIN QUERY PLAN	Inspect the access strategy chosen by the query planner
timeit	Measure median latency across multiple runs to reduce noise
Python cProfile	Function level CPU attribution for each pipeline stage
tracemalloc	Heap memory delta and peak allocation per operation

Table 4.1. Tools Used

Table	Row Count
reviews	80,000
authors	75,349
hotels	3,380

Table 4.2. Profiling Baseline

Index Name	Table	Column(s)
idx_reviews_hotel	reviews	hotel_id
idx_reviews_date	reviews	review_date
idx_reviews_hotel_date	reviews	hotel_id, review_date
ux_reviews_review_id	reviews	review_id
idx_authors_id	authors	id
ux_authors_id	authors	id

Table 4.3. List of Index

Query	Configuration	SQLite Plan
Q1: Point lookup	No index	SCAN reviews (full scan)
Q2: Date range	With index	SEARCH reviews USING INDEX idx_r_hotel (hotel_id=?)
Q3: Composite filter	With index	SEARCH reviews USING INDEX idx_r_date (review_date>?)
Q4: GROUP BY	With index	SCAN reviews USING INDEX idx_r_hotel
Q5: JOIN	With index	SCAN r + SEARCH h USING INDEX ux_h_id (hotel_id=?)
Q6: LIKE text	With index	SCAN reviews (full scan, leading wildcard)

Table 4.4. Explain Query Plan Details

Query	Access Pattern	No Index (ms)	With Index (ms)	Speedup	Row Returned	Selectivity
Q1: Point lookup	Single hotel filter	18.447	1.032	17.88×	295	0.37%
Q2: Date range	Review date >= 2012-01-01	133.088	144.095	0.92×	36,283	45.35%
Q3: Composite filter	Hotel + date range	15.097	0.768	19.66×	190	0.24%
Q4: GROUP BY	COUNT+ AVG per hotel	37.958	68.260	0.56×	3,380	4.23%
Q5: JOIN	Inner join + rating filter	62.735	50.639	1.24×	12,008	15.01%
Q6: LIKE text	Text LIKE '%excellent service%'	89.530	88.299	1.01×	1	0.001%

Table 4.5. Explain Query Plan Details

Date Cutoff	Rows Returned	Selectivity	No Index (ms)	With Index (ms)	Speedup
2010-01-01	65,316	81.6%	238.52	267.19	0.89×
2011-01-01	51,806	64.8%	185.27	208.41	0.89×
2012-01-01	36,283	45.4%	126.99	138.99	0.91×
2012-07-01	19,741	24.7%	76.25	72.71	1.05×
2012-10-01	8,586	10.7%	39.66	30.11	1.32×
2012-11-15	3,210	4.0%	23.97	11.98	1.99×
2012-12-01	1,555	1.9%	18.97	4.81	3.94×

Table 4.6. ate Range Selectivity Sweep

Index Configuration	Access Method	Median (ms)	Speedup vs No-Index
No index			
hotel_id index only			
Covering index (hotel_id, overall_rating)			

Table 4.7. Covering Index Comparison